Orchestrator Kit — Complete Usage Instructions

> Portable, project-agnostic objective-driven orchestration. Companion to `README.md`; this is the full operator reference (API, config, objective format, verification, troubleshooting).

1. What the kit is

A shell-only toolbox for running **objective-driven autonomous agent loops** with **configurable progress reporting** and **fresh-context discipline**. You point it at an objective (one small file), it dispatches the work to an agent lane (or runs checks by itself), polls until the agent is done, verifies against your own `check_cmd`, reports progress at a configured verbosity, and stops on completion — or on exactly one `[BLOCKED]` line when a human is genuinely required. It never asks "should I continue?".

Two hard design rules:

1. **Reporting is a pure observer.** Changing `ORCH_PROGRESS_LEVEL` changes *output*, never *behavior*.
1. **The framework is project-agnostic.** `core/` has no project paths, no machine paths, no owner constants. Everything project/user/machine-specific lives in adapters and reads `ORCH_*` variables from `config.env`.

Only `tmux` and `git` are required. Everything else (curl, converters, Chrome) is optional and belongs to an adapter.

2. Layout

```
orchestrator-kit/
  README.md                  quick start (this doc is the complete reference)
  config.example.env         copy to config.env and edit — ALL ORCH_* settings live h
  core/                      GENERIC — drop-in portable, do not edit for a project
    init.sh                  shared bootstrap: sets ORCH_KIT_DIR, sources config.env
    report.sh                reporting library (L0-L3 levels, 6 tags, auto-escalate/r
    poll.sh                  tmux-pane poller (--until <sha> / --idle / --timeout)
    run_loop.sh              the objective loop: guard -> dispatch -> poll -> verify -
    objective.md.template    objective-file contract (copy + fill per slice)
  adapters/                  owner/machine-specific — edit for your infrastructure
    tgsend.sh                Telegram text/doc/photo (token file + chat id from confi
    dispatch_kill_fresh.sh   remote tmux channel: kill stale session -> fresh -> laun
  examples/                  REFERENCE adapters from flutter_generator (do not keep a
    genapp.sh                approve -> generate -> validate command chain
    capture_golden.sh        one-off Flutter golden-test capture harness
    pdf_build.sh             Chrome-headless HTML->PDF + ImageMagick contact sheet
    objective.sample.md      working checks-only objective
  tools/
    md2html.js               markdown -> HTML (for HTML->PDF via pdf_build.sh)
```

3. Install (three steps)

```
# 1. Copy the kit into the target project (one project = one kit copy)
cp -R <path>/orchestrator-kit <your-project>/tools/orchestrator

# 2. Create + edit config (uncomment/override what your project needs)
cp <your-project>/tools/orchestrator/config.example.env \
   <your-project>/tools/orchestrator/config.env

# 3. Write the first objective (see core/objective.md.template)
cp <your-project>/tools/orchestrator/core/objective.md.template \
   <your-project>/objectives/my-slice.md
```

`config.env` is optional — every script falls back to sane defaults when it is absent. The defaults are the values shown in `config.example.env`.

4. Configuration reference

`config.env` is a plain shell file, `source`d by `core/init.sh`. Every script in the kit reads the `ORCH_*` variables from it (or from the environment — environment takes precedence after `config.env` is sourced, per normal shell semantics).

Variable	Default	Used by	What it controls
`ORCH_PROGRESS_LEVEL`	`L2`	`core/report.sh` (all)	Reporting verbosity `L0 < L1 < L2 < L3`; read fresh on every call
`ORCH_TG_TOKEN_FILE`	`${HOME}/.mac_companion/token`	`adapters/tgsend.sh`	File containing the Telegram bot token
`ORCH_TG_CHAT_ID`	`1117739189`	`adapters/tgsend.sh`	Target chat id for Telegram messages/attachments
`ORCH_HOST_SSH_<name>`	*(none)*	`adapters/dispatch_kill_fresh.sh`	SSH target for a remote host, e.g. `root@host.example.org`
`ORCH_HOST_OPCODE_<name>`	*(none)*	`adapters/dispatch_kill_fresh.sh`	Path to the opencode binary on that host
`ORCH_CFT_CHROME`	*(per-machine default)*	`examples/pdf_build.sh`	Headless Chrome-for-Testing binary for HTML->PDF
`ORCH_CAPTION_FONT`	`/System/Library/Fonts/Supplemental/Arial.ttf`	`examples/pdf_build.sh`	Pinned font for the ImageMagick contact-sheet

`<name>` in the host variables is the arbitrary host id you pass as `--host <name>`; each host is a single `ORCH_HOST_SSH_<name>` / `ORCH_HOST_OPCODE_<name>` pair.

5. Progress levels

Level	Reports	Use
L0	completion (`[COMPLETE]`) + blockers (`[BLOCKED]`) only	batch/cron
L1	L0 + major milestones (`[PROGRESS]`) + retries	short runs
L2 (default)	L1 + important decisions (`[DECISION]`), findings, verification results (`[VERIFY]`)	normal
L3	L2 + detailed execution/tool activity	debugging

- The knob is read on **every call**, so changing it mid-run takes effect immediately.
- **Auto-escalation:** any failure path forces L3 automatically; a clean recovery returns to the configured level. Escalation is never manual.

---

6. Standard message tags

Every emitted line starts with one of these tokens so a transcript can be grepped/split by category:

Tag	Meaning
`[PROGRESS]`	milestone reached / phase done / round started
`[DECISION]`	a decision taken (config, map, verdict, direction)
`[VERIFY]`	a verification result (test/gate/hash/diff outcome)
`[RECOVERY]`	retry/backoff/escalation-then-recovery of a failure
`[BLOCKED]`	a genuine blocker or approval gate requiring human input (stops execution)
`[COMPLETE]`	objective met (summary + artifacts)

---

7. `core/report.sh` — reporting library (source it)

```
source "$(dirname "$0")/report.sh"

report [--always] <level_needed 0-3> <TAG> <msg...>   # print iff effective level >= r
report_escalate <TAG> <msg...>                        # always prints; forces level t
report_recover  <TAG> <msg...>                        # always prints; returns to con
report_is_escalated                                   # exit 0 while escalated, else 1
```

- Levels are numeric (`0<=L0<L1<L2<L3`), so one message can target several levels with the same call.
- `report --always 0 ...` prints unconditionally — used for `[COMPLETE]` / `[BLOCKED]` so they are never hidden even on a quiet level.
- `report_escalate` / `report_recover` always print: their job is guaranteeing failure-path visibility even under L0.

---

8. `core/poll.sh` — tmux-pane poller

```
poll.sh --lane <tmux-session> [--until <sha>] [--idle] [--timeout <mins>] [--quiet] [-
```

Flag	Behavior
`--lane <session>` (required)	tmux session to poll
`--until <sha>`	success once `<sha>` appears in the pane's visible text
`--idle`	success once the pane's last non-blank line ends with an idle `❯` prompt
`--timeout <m>`	minutes before giving up (default 10)

Flag	Behavior
`--quiet`	suppress the status line (exit code only)
`--report`	also print the last meaningful pane line

Exit codes: `0` success · `1` timeout or a visible `FAIL` / `Error:` / `Exception` line · `2` bad or missing tmux session.

If both `--until` and `--idle` are given, success requires the SHA to have appeared **and** the lane to now be idle (commit landed, agent done).

Idle detection caveat: a bare `❯` alone is *not* reliable — busy Claude Code panes keep a persistent input box with `❯` plus a `thought for Ns)` spinner, so idle is defined as `❯` visible **and** no spinner line. Remote opencode-flavored panes may render differently; if idle never triggers there, this regex needs a lane-specific update (documented in the script header).

---

9. `core/run_loop.sh` — the objective loop

```
run_loop.sh --objective <file.md> [--level L2] [--lane <tmux-session>] [--from <SHA>]
            [--until-commit <SHA>] [--max-rounds N] [--timeout <mins>] [--repo <dir>]
```

Flag	Default	What it does
`--objective <file>`	required	the objective file (see §10)
`--level Ln`	objective's `level` field, else `L2`	reporting verbosity
`--lane <tmux>`	objective's `lane` field	tmux session to dispatch the prompt into; empty = checks-only run
`--from <SHA>`	objective's `from`, else current HEAD	starting commit for round tracking
`--until-commit <SHA>`	objective's `until_commit`	complete only once `check_cmd` passes at this commit
`--max-rounds N`	objective's `max_rounds`, else `10`	hard stop (becomes `[BLOCKED]`)
`--timeout <mins>`	objective's `timeout_mins`, else `10`	max minutes to wait for the lane to go idle per round
`--repo <dir>`	git root of the current working directory	which git repo to verify against

CLI flags override the matching objective-file field when both are given. Only `check_cmd` in the objective file is mandatory.

Round protocol (per round `n`, up to `max_rounds`)

1. **Guard** — if `--until-commit` already equals HEAD, emit `[COMPLETE] objective already satisfied` and exit 0 before doing anything.
1. **Dispatch** — if a `lane` is configured, `tmux send-keys` the objective's `prompt` into it; a missing/dead lane is an immediate `[BLOCKED]`. No lane = verification-only run.
1. **Poll** — `poll.sh --lane --idle` with the round timeout. Timeouts escalate to L3 and retry

with backoff (`1s → up to 10s` per round).

1. **Verify** — run `check_cmd` inside `--repo` ; capture output to a temp log.

- PASS: `[VERIFY]` , recover verbosity if it was escalated, update `current_head` .
- FAIL: escalate to L3 (`[RECOVERY] … escalating`), print the last log line, backoff, retry.

1. **Complete** — if `--until-commit` is set, only finish when `check_cmd` passes at that exact

HEAD (`[COMPLETE] objective met at <sha>`); otherwise finish as soon as `check_cmd` passes (first round that is green).

The loop halts on exactly two conditions: `[COMPLETE]` (objective met) or one `[BLOCKED]` (approval gate, dead lane, `max_rounds` exhausted). There are no "should I continue?" prompts.

---

10. Objective-file format

Plain `key: value` lines (one value per line; shell-greppable, no YAML/JSON parser). Lines starting with `#` and blank lines are ignored. Only `check_cmd` is required.

Field	Required	Meaning
`goal`	no	one-line statement of what "done" means
`acceptance`	no	the exact gate(s) a human would check to call this done
`check_cmd`	**yes**	command proving the objective is met; run inside `--repo` each round, e.g. `npm run typecheck:builder && npx jest test/s1_visual_intent.test.ts`
`artifacts`	no	evidence/artifacts the objective is expected to produce
`lane`	no	tmux session to dispatch into; empty = checks-only, no dispatch
`from`	no	SHA the run starts from
`until_commit`	no	SHA marking completion once `check_cmd` passes there; empty = complete on the first green round
`level`	no	reporting level (`L0` – `L3`), default `L2`
`max_rounds`	no	max rounds before `[BLOCKED]` , default `10`
`timeout_mins`	no	lane-idle wait per round, default `10`
`prompt`	no	single-line prompt/command sent into the lane each round via `tmux send-keys`

Template: `core/objective.md.template` . Worked example: `examples/objective.sample.md` .

---

11. Adapters

`adapters/tgsend.sh` — Telegram

```
tgsend.sh text  "message"
tgsend.sh doc   <file> ["caption"]    # sendDocument — preferred for files/reports
tgsend.sh photo <file> ["caption"]    # sendPhoto — screenshots/goldens
```

Reads `ORCH_TG_TOKEN_FILE` (bot token) and `ORCH_TG_CHAT_ID` from `config.env` . Prints nothing on success, an error line + non-zero exit on failure. Never used for heavy back-and-forth — one point/paragraph per `text` message, files as `doc` attachments.

`adapters/dispatch_kill_fresh.sh` — remote tmux channel

```
dispatch_kill_fresh.sh --host <name> --channel <name> --workdir <path> [--primed] --y
```

Looks up `ORCH_HOST_SSH_<name>` / `ORCH_HOST_OPCODE_<name>` in `config.env`, then: kill the existing remote tmux session → start a fresh detached session in `<workdir>` → launch opencode → dismiss a first-launch welcome overlay if seen → wait for a ready prompt. It does **not** send the task prompt — `run_loop.sh` (or you) sends that separately with `tmux send-keys`.

Destructive — kills a remote tmux session, so it refuses to run without `--yes`; without `--yes` it prints the planned actions (dry run). Use this for every new task: a fresh channel is cheap and carries no polluted context from the previous one.

---

12. Writing a new adapter (from `examples/`)

1. `source core/init.sh` so it picks up `config.env`.
2. Read machine/user-specific values with `: "${VAR:=default}"` (never hardcode).
3. Do exactly one task; print one line per meaningful result; exit non-zero on failure.
4. If the task's verbosity should react to the level, also `source core/report.sh` and use the

standard tags.

Reference adapters shipped in `examples/` (project-specific, do not keep as-is): `genapp.sh` (approve→generate→validate for a generated app), `capture_golden.sh` (one-off Flutter golden harness), `pdf_build.sh` (Chrome HTML→PDF + ImageMagick contact sheet).

---

13. End-to-end worked example

Objective `objectives/s6-slice5.md` for a "write the A11y test generator" slice:

```
goal: land the A11y test generator slice (regression suite green)
acceptance: typecheck clean, jest green, validate.ts VALIDATION PASSED on all samples
check_cmd: npm run typecheck:builder && npx jest test/s1_visual_intent.test.ts && git
artifacts: commit SHA + test/s1_visual_intent.test.ts additions
lane: s-hermetic
from: c848640
until_commit: f030dd4
level: L2
max_rounds: 8
timeout_mins: 15
prompt: read design/flutter-app-builder/research/S6_IMPL_BRIEF_CLAUDE.md, implement s
```

Run:

```
tools/orchestrator/run_loop.sh --objective objectives/s6-slice5.md
```

What you see at L2: `[PROGRESS]` round starts, `[VERIFY]` pass/fail per round, `[RECOVERY]` on any failure with auto-escalation, `[COMPLETE] objective met at f030dd4` when the until-commit lands green. If the lane dies or rounds run out: exactly one `[BLOCKED]` line, then it stops.

A checks-only run (no lane, no agent) reuses the same loop to watch any external process reach a commit — see `examples/objective.sample.md`.

---

14. Verification (prove the kit works)

1. **Level gating** — run the same objective at `--level L0` and `--level L3`; outcomes are identical, output radically different (L0 ≈ 2-4 lines, L3 verbose).
1. **Auto-escalate/recover** — make one `check_cmd` fail; observe `[RECOVERY]` emitted at verbose level, then a clean success returns to the configured level.
1. **Tags** — every emitted line begins with one of the six tags; grep the transcript for the tag histogram.
1. **Autonomy** — a generated run transcript contains no "should I / continue? / permission?" prompts; it runs to `[COMPLETE]` (or one `[BLOCKED]`) unaided.
1. **Gates intact** — your project's own check commands still pass (run_loop never mutates the repo; it only reads git and runs your `check_cmd`).

Quick sanity:

```
bash -n tools/orchestrator/core/*.sh tools/orchestrator/adapters/*.sh   # no syntax e
tools/orchestrator/core/run_loop.sh --help                              # usage print
source tools/orchestrator/core/report.sh; report 0 "[TEST]" "hello"     # prints at a
```

15. Troubleshooting

Symptom	Cause / fix
`lane '<name>' has no live tmux session` → `[BLOCKED]`	the lane isn't running. Start it (`dispatch_kill_fresh.sh --host … --yes` for remote, or attach locally) and re-run
Lane never goes idle → timeouts, escalating	`poll.sh` idle detection is tuned for Claude Code TUI panes; remote opencode may need a `--until <sha>` target or a lane-specific idle regex (see `poll.sh` header)
`[BLOCKED] objective not met after N round(s)`	`max_rounds` exhausted. Increase it, fix the lane, or split the objective into smaller ones
`--repo is not a git repo`	`--repo` must point at a git repo (or a dir whose git root is discoverable); run_loop defaults to the cwd's git root
check_cmd keeps failing	read the last log line printed with the `[RECOVERY]` message; fix the objective/project, not the kit
no config found but scripts complain about missing vars	copy `config.example.env` → `config.env` and set the required `ORCH_*` variables
tgsend fails with HTTP ≠ 200	token file missing (`ORCH_TG_TOKEN_FILE`), wrong chat id, or the bot can't reach the chat

16. Loop discipline (how to keep looping without burning context)

- One objective = one fresh session. Pass `objective.md` file paths, never conversation history.
- Repository artifacts (briefs, reports, tests, `HANDOFF.md`) are the durable state; the session is disposable.

- Between independent objectives: `/clear` (local) or kill+fresh the remote channel

(`dispatch_kill_fresh.sh --yes`), then re-anchor on the objective file.

- Mid-slice with growing context: `/compact`, keep working.
- `[COMPLETE]` is the trigger to stop, checkpoint/commit, and start the next objective fresh.